



Infrastructure PKI sous Linux

Autorité de Certification & Gestion des
Certificats

Sean Fritsch

INTRODUCTION

L'infrastructure à clés publiques (PKI - Public Key Infrastructure) constitue le socle de confiance de toute architecture réseau sécurisée. Elle permet d'émettre, de distribuer et de révoquer des certificats numériques garantissant l'identité des machines et des utilisateurs au sein d'un système d'information. Ce document présente la mise en place complète d'une PKI privée sous Linux à l'aide d'OpenSSL, depuis la création de l'Autorité de Certification (CA) racine jusqu'à la génération et la signature de certificats serveurs, en passant par l'automatisation de ces opérations via un script Bash.

PRÉREQUIS

- Système d'exploitation : Ubuntu Server 22.04 / Debian 12.
- Paquet requis : `openssl` (installé par défaut sur la plupart des distributions).
- Accès : Privilèges Sudo.
- Réseau : Connectivité entre le serveur CA et les machines clientes.

I. RAPPELS THÉORIQUES

1. Principe de la cryptographie asymétrique

La cryptographie asymétrique repose sur une paire de clés mathématiquement liées : une **clé privée** (secrète, détenue uniquement par le propriétaire) et une **clé publique** (diffusée librement). Un message chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante, et inversement. Ce mécanisme est à la base de la signature numérique et du chiffrement TLS/SSL.

2. Composants d'une PKI

- **Autorité de Certification (CA)** : Entité de confiance qui signe les certificats. Elle garantit l'identité du demandeur.
- **Certificat numérique (X.509)** : Document électronique liant une clé publique à une identité (nom de domaine, organisation). Il contient la signature de la CA.
- **CSR (Certificate Signing Request)** : Requête de signature envoyée à la CA. Elle contient la clé publique du demandeur et ses informations d'identité.
- **CRL (Certificate Revocation List)** : Liste des certificats révoqués avant leur date d'expiration, publiée par la CA.

3. Chaîne de confiance

La confiance dans un certificat repose sur une chaîne hiérarchique : le certificat serveur est signé par une CA intermédiaire, elle-même signée par la CA racine. Le client (navigateur, système) vérifie cette chaîne en remontant jusqu'à une CA racine présente dans son magasin de certificats de confiance (trust store). Dans le cadre de ce Lab, nous utilisons une CA racine auto-signée.

II. MISE EN PLACE DE LA CA RACINE

1. Création de l'arborescence

Une structure de répertoires organisée est indispensable pour gérer proprement les certificats, les clés et les fichiers d'index de la CA.

```
# Création de l'arborescence de la CA
sudo mkdir -p /etc/pki/CA/{certs,crl,newcerts,private,csr}
sudo chmod 700 /etc/pki/CA/private

# Initialisation des fichiers d'index
sudo touch /etc/pki/CA/index.txt
echo 1000 | sudo tee /etc/pki/CA/serial
echo 1000 | sudo tee /etc/pki/CA/crlnumber
```

Le répertoire `private/` est restreint en permissions (700) car il contiendra la clé privée de la CA, élément le plus sensible de toute l'infrastructure.

2. Génération de la clé privée de la CA

```
# Générer une clé RSA 4096 bits protégée par passphrase
sudo openssl genrsa -aes256 \
  -out /etc/pki/CA/private/ca.key.pem 4096

# Sécurisation des permissions
sudo chmod 400 /etc/pki/CA/private/ca.key.pem
```

L'option `-aes256` chiffre la clé privée avec une passphrase. Cette protection est critique : si la clé de la CA est compromise, l'ensemble de l'infrastructure de confiance est invalidée.

3. Création du certificat racine auto-signé

```
# Générer le certificat racine (validité 10 ans)
sudo openssl req -new -x509 -days 3650 \
  -key /etc/pki/CA/private/ca.key.pem \
  -sha256 \
  -out /etc/pki/CA/certs/ca.cert.pem \
  -subj "/C=FR/ST=IDF/L=Courbevoie/O=BTS-SIO-Lab"\
  "/OU=SISR/CN=CA-Racine-Lab"
```

Détail des champs du **Distinguished Name (DN)** :

Champ	Signification	Valeur
C	Country	FR
ST	State	IDF
L	Locality	Courbevoie
O	Organization	BTS-SIO-Lab
OU	Organizational Unit	SISR
CN	Common Name	CA-Racine-Lab

III. GÉNÉRATION D'UN CERTIFICAT SERVEUR

1. Génération de la clé privée du serveur

```
# Clé RSA 2048 bits pour le serveur (sans passphrase pour les services)
openssl genrsa -out /etc/pki/CA/private/serveur.key.pem 2048
chmod 400 /etc/pki/CA/private/serveur.key.pem
```

Contrairement à la CA, la clé du serveur n'est pas protégée par passphrase afin de permettre le redémarrage automatique des services (Apache, Nginx) sans intervention manuelle.

2. Création de la requête de signature (CSR)

```
# Générer le CSR pour le serveur web
openssl req -new \
  -key /etc/pki/CA/private/serveur.key.pem \
  -out /etc/pki/CA/csr/serveur.csr.pem \
  -subj "/C=FR/ST=IDF/L=Courbevoie/O=BTS-SIO-Lab" \
  "/OU=SISR/CN=srv-web.lab.local"
```

Le **Common Name (CN)** doit correspondre exactement au nom de domaine (FQDN) par lequel les clients accéderont au serveur. Une incohérence provoquera une erreur de certificat côté client.

3. Signature du certificat par la CA

```
# La CA signe le CSR et délivre le certificat (validité 1 an)
sudo openssl x509 -req -days 365 \
  -in /etc/pki/CA/csr/serveur.csr.pem \
  -CA /etc/pki/CA/certs/ca.cert.pem \
  -CAkey /etc/pki/CA/private/ca.key.pem \
  -CAcreateserial \
  -out /etc/pki/CA/certs/serveur.cert.pem \
  -sha256
```

4. Vérification du certificat

```
# Vérifier la chaîne de confiance
openssl verify -CAfile /etc/pki/CA/certs/ca.cert.pem \
  /etc/pki/CA/certs/serveur.cert.pem

# Lire le contenu du certificat
openssl x509 -in /etc/pki/CA/certs/serveur.cert.pem \
  -text -noout
```

La commande `openssl verify` doit retourner **OK**. Si elle retourne une erreur, la chaîne de confiance est rompue et le certificat sera rejeté par les clients.

IV. AUTOMATISATION VIA SCRIPT BASH

1. Présentation du script deploy_pki.sh

Le script suivant automatise l'intégralité du déploiement de la PKI : création de l'arborescence, génération de la CA racine, et émission d'un certificat serveur. Il intègre des contrôles d'erreur à chaque étape critique.

2. Code du Script

```
#!/bin/bash
# =====
# deploy_pki.sh - Déploiement automatisé d'une PKI
# Auteur : Sean Fritsch | BTS SIO SISR
# =====

set -e # Arrêt immédiat en cas d'erreur

# --- 1. Variables ---
CA_DIR="/etc/pki/CA"
DAYS_CA=3650          # Validité CA : 10 ans
DAYS_CERT=365         # Validité certificat : 1 an
KEY_SIZE_CA=4096
KEY_SIZE_SRV=2048
FQDN="srv-web.lab.local"

echo "[*] Déploiement de la PKI en cours..."

# --- 2. Création de l'arborescence ---
echo "[1/5] Création de l'arborescence..."
sudo mkdir -p $CA_DIR/{certs,crl,newcerts,private,csr}
sudo chmod 700 $CA_DIR/private
sudo touch $CA_DIR/index.txt
echo 1000 | sudo tee $CA_DIR/serial > /dev/null

# --- 3. Génération de la CA racine ---
echo "[2/5] Génération de la clé privée CA..."
sudo openssl genrsa -aes256 \
    -out $CA_DIR/private/ca.key.pem $KEY_SIZE_CA

sudo chmod 400 $CA_DIR/private/ca.key.pem

echo "[3/5] Création du certificat racine..."
sudo openssl req -new -x509 -days $DAYS_CA \
    -key $CA_DIR/private/ca.key.pem \
    -sha256 \
    -out $CA_DIR/certs/ca.cert.pem \
    -subj "/C=FR/ST=IDF/L=Courbevoie/O=BTS-SIO-Lab\
/OU=SISR/CN=CA-Racine-Lab"

# --- 4. Génération du certificat serveur ---
echo "[4/5] Génération de la clé et du CSR serveur..."
sudo openssl genrsa \
    -out $CA_DIR/private/serveur.key.pem $KEY_SIZE_SRV
sudo chmod 400 $CA_DIR/private/serveur.key.pem

sudo openssl req -new \
```

```
-key $CA_DIR/private/serveur.key.pem \  
-out $CA_DIR/csr/serveur.csr.pem \  
-subj "/C=FR/ST=IDF/L=Courbevoie/O=BTS-SIO-Lab\  
/OU=SISR/CN=$FQDN"  
  
# --- 5. Signature et vérification ---  
echo "[5/5] Signature du certificat serveur..."  
sudo openssl x509 -req -days $DAYS_CERT \  
-in $CA_DIR/csr/serveur.csr.pem \  
-CA $CA_DIR/certs/ca.cert.pem \  
-CAkey $CA_DIR/private/ca.key.pem \  
-CAcreateserial \  
-out $CA_DIR/certs/serveur.cert.pem -sha256  
  
# Vérification de la chaîne  
echo "[*] Vérification de la chaîne de confiance..."  
openssl verify -CAfile $CA_DIR/certs/ca.cert.pem \  
$CA_DIR/certs/serveur.cert.pem  
  
if [ $? -eq 0 ]; then  
    echo "[OK] PKI déployée avec succès !"  
    echo "   CA      : $CA_DIR/certs/ca.cert.pem"  
    echo "   Serveur : $CA_DIR/certs/serveur.cert.pem"  
else  
    echo "[ERREUR] Chaîne de confiance invalide !"  
    exit 1  
fi
```

V. RÉVOCATION D'UN CERTIFICAT

La révocation est une opération critique qui invalide un certificat avant sa date d'expiration. Elle est nécessaire en cas de compromission de la clé privée, de changement de personnel, ou de décommissionnement d'un serveur.

1. Révoquer un certificat

```
# Révoquer le certificat serveur  
sudo openssl ca -revoke /etc/pki/CA/certs/serveur.cert.pem \  
-keyfile /etc/pki/CA/private/ca.key.pem \  
-cert /etc/pki/CA/certs/ca.cert.pem
```

2. Générer la CRL (Certificate Revocation List)

```
# Publier la liste de révocation mise à jour  
sudo openssl ca -gencrl \  
-keyfile /etc/pki/CA/private/ca.key.pem \  
-cert /etc/pki/CA/certs/ca.cert.pem \  
-out /etc/pki/CA/crl/ca.crl.pem
```

La CRL doit être publiée et accessible aux clients pour qu'ils puissent vérifier qu'un certificat n'a pas été révoqué. Dans un environnement de production, cette vérification peut également se faire via le protocole OCSP (Online Certificate Status Protocol).

VI. VÉRIFICATION ET TESTS

1. Lecture du certificat

```
# Afficher les détails du certificat serveur
openssl x509 -in /etc/pki/CA/certs/serveur.cert.pem \
  -text -noout | head -20
```

2. Test avec un serveur HTTPS local

```
# Lancer un serveur HTTPS de test avec OpenSSL
sudo openssl s_server \
  -cert /etc/pki/CA/certs/serveur.cert.pem \
  -key /etc/pki/CA/private/serveur.key.pem \
  -CAfile /etc/pki/CA/certs/ca.cert.pem \
  -accept 443 -www

# Depuis un client : tester la connexion TLS
openssl s_client -connect srv-web.lab.local:443 \
  -CAfile /etc/pki/CA/certs/ca.cert.pem
```

- **Succès attendu** : La commande `s_client` affiche `Verify return code: 0 (ok)`.
- **Échec attendu** : Sans le certificat CA dans le trust store, le client retourne `verify error:num=19:self signed certificate in certificate chain`.

ANNEXES : COMMANDES UTILES

Commande	Fonction
<code>openssl genrsa -aes256 -out key.pem 4096</code>	Générer une clé RSA chiffrée
<code>openssl req -new -x509 -key ca.key -out ca.cert</code>	Créer un certificat auto-signé
<code>openssl req -new -key srv.key -out srv.csr</code>	Générer une requête de signature (CSR)
<code>openssl x509 -req -CA ca.cert -CAkey ca.key</code>	Signer un CSR avec la CA
<code>openssl verify -CAfile ca.cert srv.cert</code>	Vérifier la chaîne de confiance
<code>openssl x509 -in cert.pem -text -noout</code>	Lire le contenu d'un certificat
<code>openssl ca -revoke cert.pem</code>	Révoquer un certificat
<code>openssl ca -gencrl -out ca.crl.pem</code>	Générer la liste de révocation (CRL)
<code>openssl s_client -connect host:443</code>	Tester une connexion TLS

CONCLUSION

La mise en place de cette infrastructure PKI sous Linux démontre la maîtrise du cycle de vie complet des certificats numériques : génération des clés, création d'une Autorité de Certification racine, émission de certificats serveurs, et processus de révocation. Ce Lab couvre les fondamentaux indispensables à tout administrateur réseau et systèmes évoluant dans un environnement sécurisé.

L'apport majeur de ce projet réside dans le script **deploy_pki.sh**, qui garantit :

- **La Reproductibilité** : L'automatisation complète du déploiement permet de recréer l'infrastructure de manière identique sur n'importe quel serveur Linux, éliminant les erreurs manuelles.
- **La Sécurité par conception** : Les permissions restrictives (`chmod 400/700`) sur les clés privées et le chiffrement AES-256 de la clé CA appliquent le principe du moindre privilège dès l'initialisation.
- **La Traçabilité** : L'utilisation des fichiers d'index et de numéros de série permet un suivi rigoureux de chaque certificat émis ou révoqué.

Le choix d'une clé RSA 4096 bits pour la CA et 2048 bits pour les certificats serveurs offre un compromis optimal entre robustesse cryptographique et performance. Ce projet constitue une base solide pour une évolution vers une PKI hiérarchique à deux niveaux (CA racine + CA intermédiaire), architecture recommandée en environnement de production.

***Perspective d'évolution** : L'intégration de cette PKI avec un serveur LDAP ou Active Directory permettrait la distribution automatique des certificats via les stratégies de groupe (GPO), tandis que le couplage avec un répondeur OCSP remplacerait la publication manuelle des CRL par une vérification en temps réel du statut des certificats.*